

Chapter 14

Kubernetes in Action 1st Edition

Computational Resources Management

Pod 리소스 관리 — requests·limits·QoS·HPA·VPA·KEDA



신재근 (Jagger)

✓ 실측 검증됨 · minikube v1.38.1 + metrics-server + KEDA v2.16 · K8s v1.35.1 · 2026-06-01

왜 14장이 중요한가 — 자원 관리는 "보안"이다

자원이 충돌하면 어떻게 되는가

- Noisy neighbor: 한 Pod이 노드 CPU·메모리를 독점 → 옆 Pod이 throttle/OOM
- Schedule 실패: requests가 잘못되면 Pod이 절대 못 뜨거나 노드를 과적
- DoS의 안쪽 면: 보안 침해보다 자원 미설정으로 인한 장애가 더 잦음
- 비용 폭증: 클라우드에서 limits 없는 Pod 하나가 노드 한 대 청구서를 만든다

14장의 답 — 4축 격벽

1. requests / limits — 컨테이너 단위 자원 한도
2. QoS classes — 자원 부족 시 종료 순서
3. LimitRange / ResourceQuota — namespace 단위 정책
4. HPA / VPA / KEDA — 자동 스케일링 (수평·수직·이벤트)

책 1판(2018) vs 현행(2026) — 큰 변경점

Heapster (책)는 K8s 1.13에서 제거됨

- 책 14장 메트릭은 `Heapster` 가정 → 현행은 `metrics-server` 표준
- `kubectl top` 명령은 그대로지만 데이터 소스가 바뀜

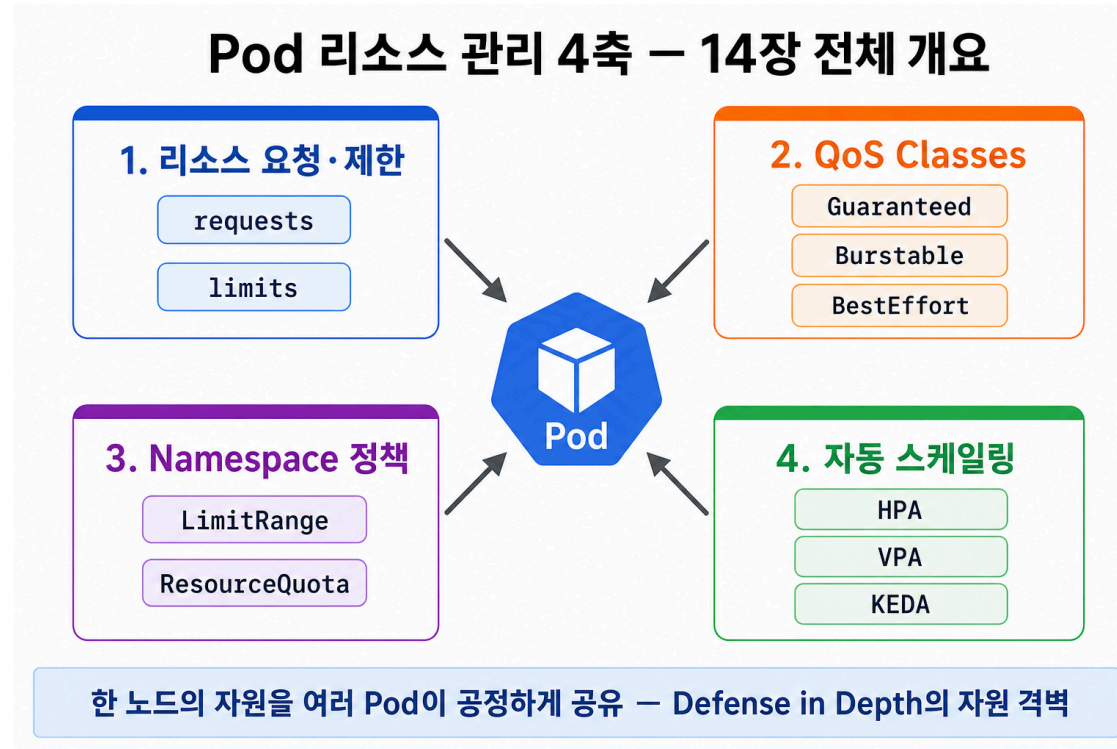
HPA v1 (책) → HPA v2 (현행 표준)

- 책: `autoscaling/v1` — CPU 평균만
- 현행 표준: `autoscaling/v2` — CPU + Memory + Custom + External + behavior 정책

책에 없는 것 — 추가 학습 대상

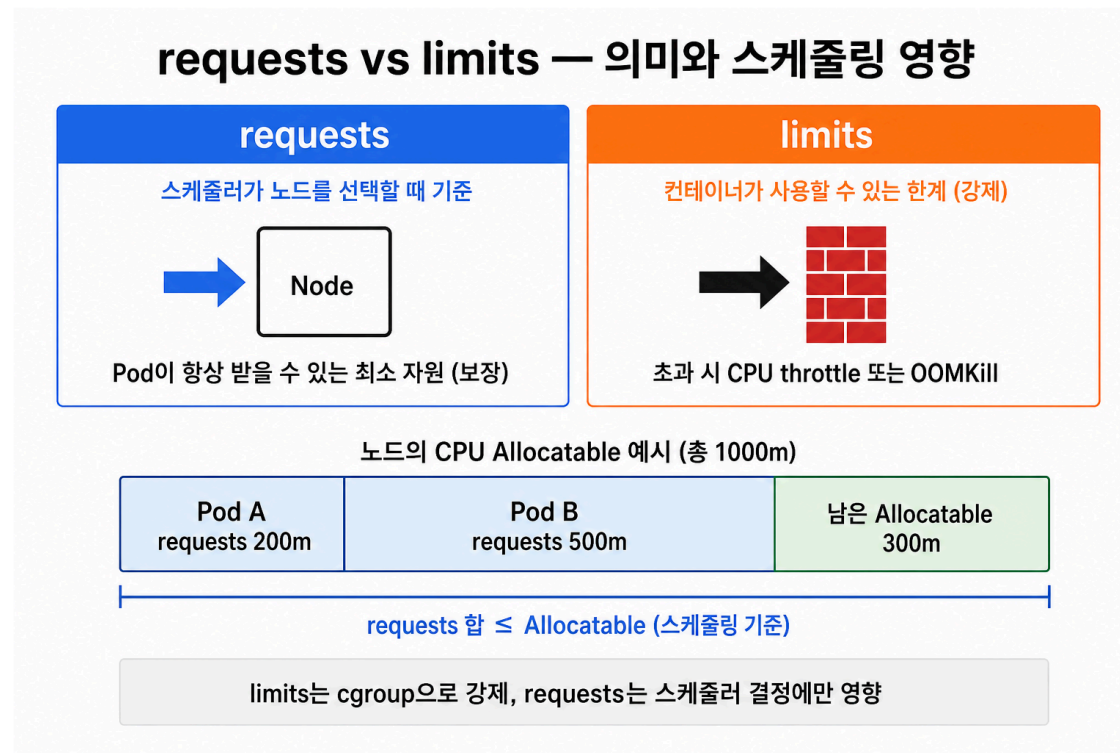
- VPA (수직 스케일러) — Pod 자체 크기를 조정
- KEDA (이벤트 기반 스케일러) — `0 → N` replica 가능 (HPA 한계 우회)
- In-place Pod Resize (1.32 beta) — 재시작 없이 limits 변경
- Pod-level resources (1.32 alpha) — 사이드카 시대의 자원 표현
- MemoryQoS (cgroup v2, 1.27+) — 메모리 압박 시 더 정교한 reclaim

14장의 4축 — 한 장 요약



각 축이 작동하는 레벨이 모두 다름 — 컨테이너 spec(requests/limits), Pod 통합 평가(QoS), namespace label/quota(LimitRange/ResourceQuota), 외부 컨트롤러(HPA/VPA/KEDA). 겹쳐 적용해야 의미 있는 자원 격벽.

14.1 requests vs limits — 의미와 영향



requests는 스케줄링 결정에만 영향, limits는 cgroup으로 강제. requests 합 ≤ Allocatable이 노드 선택 기준이고, limits 초과는 CPU throttle 또는 OOMKill로 즉시 강제됨.

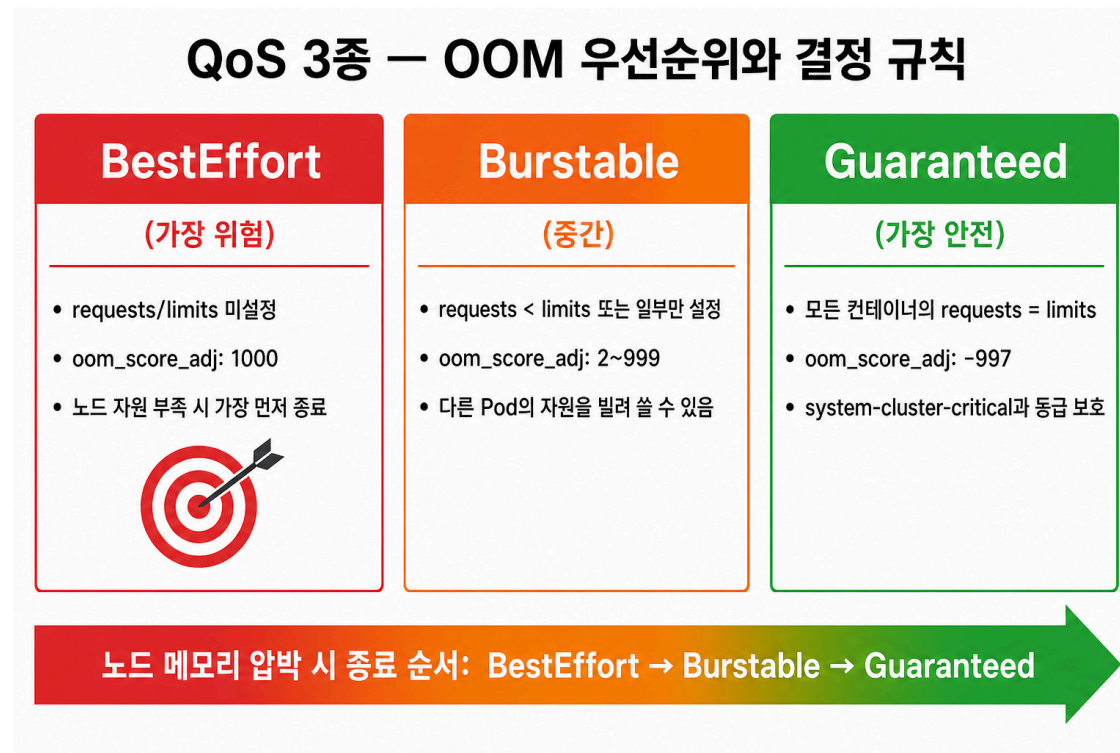
Demo 1 — requests/limits 적용 (실측)

```
# 01-resource-requests-limits.yaml
spec:
  containers:
  - name: main
    image: busybox:stable
    resources:
      requests: { cpu: "200m", memory: "100Mi" }
      limits: { cpu: "400m", memory: "200Mi" }
```

```
$ kubectl describe pod resource-pod | grep -A 5 "Limits\|Requests"
Limits:
  cpu:      400m
  memory:   200Mi
Requests:
  cpu:      200m
  memory:   100Mi
```

- requests 200m: 스케줄러가 이만큼 여유 있는 노드 선택
- limits 400m: cgroup이 강제 — 초과 사용 시 throttle

14.2 QoS Classes — OOM 우선순위



자동 결정 규칙: 모든 컨테이너에서 requests = limits → Guaranteed, 일부만 설정 또는 requests < limits → Burstable, 둘 다 미설정 → BestEffort. 운영 표준은 Guaranteed (보안과 동일 원칙 — "명시적으로").

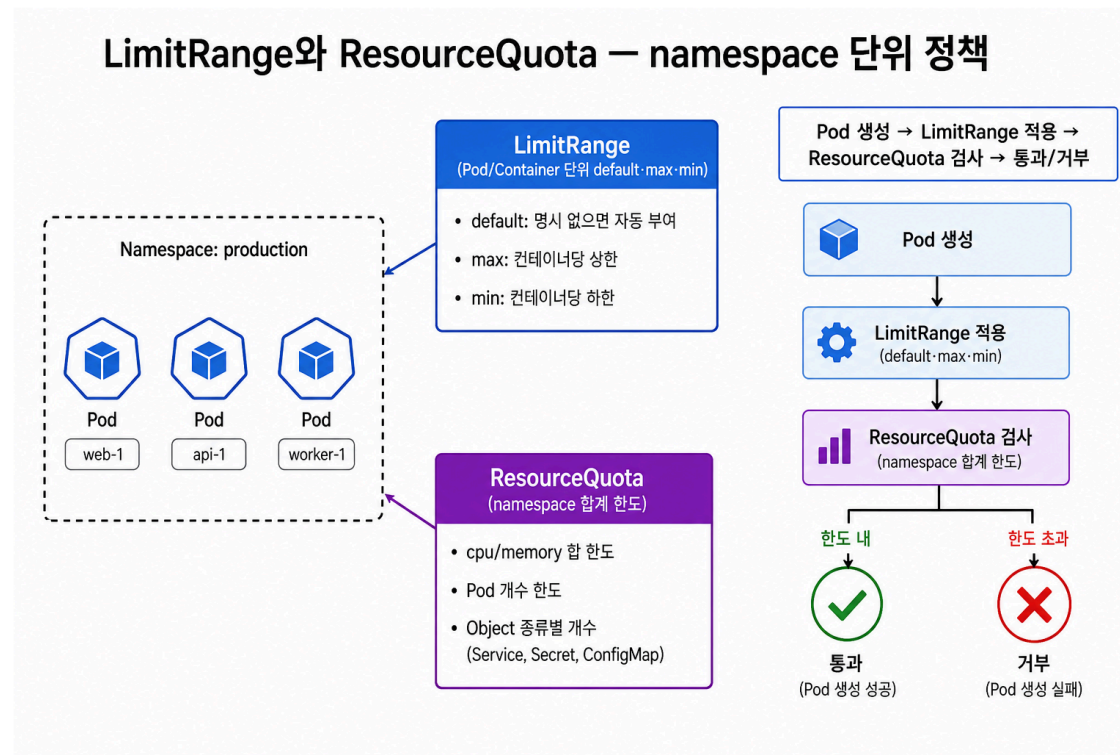
Demo 2 — QoS 3종 동시 실행 (실측)

```
# 02-qos-classes.yaml - 3개 Pod 동시 적용
qos-guaranteed: requests = limits = { cpu: 100m, memory: 64Mi }
qos-burstable:  requests = 100m/64Mi, limits = 200m/128Mi
qos-besteffort: requests/limits 모두 미설정
```

```
$ for p in qos-guaranteed qos-burstable qos-besteffort; do
    qos=$(kubectl get pod $p -o jsonpath='{.status.qosClass}')
    echo "$p → $qos"
done
qos-guaranteed → Guaranteed
qos-burstable   → Burstable
qos-besteffort  → BestEffort
```

spec에 `qosClass` 필드는 없음 — K8s가 requests/limits로부터 자동 계산

14.3 LimitRange + ResourceQuota — namespace 정책



두 객체의 역할 분리: LimitRange = Pod/Container 단위 default·max·min, ResourceQuota = namespace 합계 한도. 둘 다 admission 단계에서 강제 — 위반 시 즉시 거부.

Demo 3 — LimitRange 자동 부여 (실측)

```
# 03-limitrange-default.yaml
spec:
  limits:
  - type: Container
    default:      { cpu: "200m", memory: "128Mi" }    # limits 자동 부여
    defaultRequest: { cpu: "100m", memory: "64Mi" }    # requests 자동 부여
    max:          { cpu: "500m", memory: "512Mi" }
    min:          { cpu: "50m",  memory: "32Mi"  }
```

```
# resources를 명시 안 한 Pod 적용 후 자동 부여 확인
$ kubectl describe pod -n limitrange-demo no-resources-pod | grep -A 4 "Limits\|Requests"
Limits:
  cpu:      200m      # ← LimitRange의 default가 자동 부여
  memory:   128Mi
Requests:
  cpu:      100m      # ← LimitRange의 defaultRequest가 자동 부여
  memory:   64Mi
```

Demo 4 — ResourceQuota 한도 초과 거부 (실측)

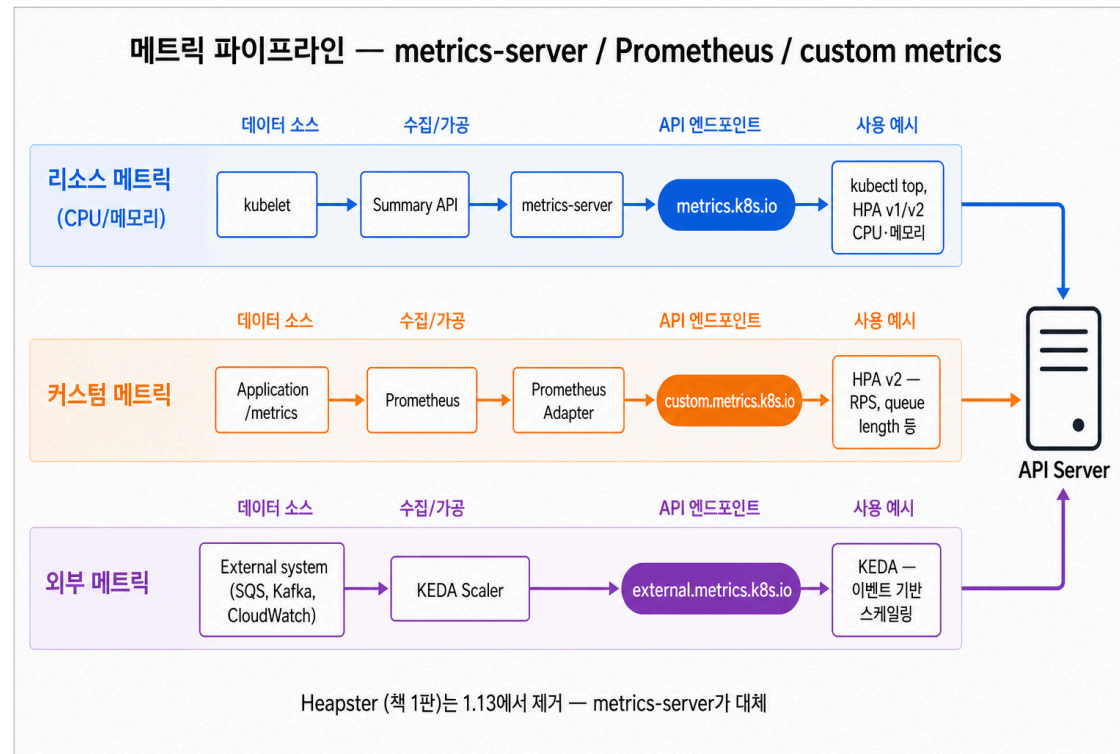
```
# 04-resourcequota-cap.yaml
spec:
  hard:
    requests.cpu: "500m"
    requests.memory: "512Mi"
    pods: "5"
```

```
# 한도 초과 Pod 적용 시도
$ kubectl apply -n quota-demo -f - <<EOF
spec:
  containers: [{ resources: { requests: { cpu: "500m" } } }]
EOF

Error from server (Forbidden): pods "quota-overrun" is forbidden:
  exceeded quota: ns-quota,
  requested: requests.cpu=500m,
  used:      requests.cpu=100m,
  limited:   requests.cpu=500m
```

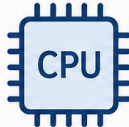
admission 단계에서 즉시 거부 — Pod이 만들어지지 않음. 13장 PSA 거부와 같은 패턴.

14.4 메트릭 파이프라인 — Heapster의 죽음



Heapster는 K8s 1.13에서 제거됨 — 책 14장의 메트릭 명령은 그대로지만 데이터 소스가 metrics-server로 바뀜. HPA v2는 3개 API(metrics.k8s.io / custom.metrics.k8s.io / external.metrics.k8s.io)에서 동시에 메트릭을 가져올 수 있다.

14.5 HPA v1 vs v2 — 책의 가장 큰 코드 변경점

HPA v1 vs v2 — 단일 CPU에서 다중 메트릭으로	
HPA v1 (autoscaling/v1) — 책 1판	HPA v2 (autoscaling/v2) — 현재
• CPU 평균 사용률만 	 CPU  Memory  Custom (RPS, queue)  External (SQS depth)
단일 메트릭, 단순한 비례 계산, behavior 정책 없음	다중 메트릭 동시 평가 (max), behavior 정책으로 scale up/down 속도 분리 제어, ContainerResource 메트릭 (1.27+)
예시 YAML (비교) — HPA v1	예시 YAML (비교) — HPA v2
<pre>metrics: [- type: Resource resource: cpu targetAverageUtilization: 50]</pre>	<pre>metrics: [- type: Resource - type: Pods - type: External]</pre>
1판은 v1으로 작성됨 — v2가 GA된 1.23부터 표준	

책의 HPA YAML은 v1 — 현재 클러스터에서 동작하지만 단일 메트릭만 됨. v2의 핵심은 (1) 다중 메트릭 동시 평가, (2) **behavior** 정책으로 scale up/down 속도 분리 제어, (3) ContainerResource 메트릭으로 sidecar 영향 제거.

Demo 5 — HPA v2 + CPU 부하 (실측 1/2)

```
# 05-hpa-v2-cpu.yaml - autoscaling/v2 + behavior 정책
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { kind: Deployment, name: cpu-stress }
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource: { name: cpu, target: { type: Utilization, averageUtilization: 50 } }
  behavior:
    scaleUp: { stabilizationWindowSeconds: 0, policies: [{ type: Percent, value: 100, periodSeconds: 15 }] }
    scaleDown: { stabilizationWindowSeconds: 300, policies: [{ type: Percent, value: 50, periodSeconds: 60 }] }
```

```
$ kubectl apply -f 05-hpa-v2-cpu.yaml           # vish/stress 컨테이너로 1 vCPU 부하
$ kubectl get hpa cpu-stress-hpa
```

NAME	TARGETS	MINPODS	MAXPODS	REPLICAS	
cpu-stress-hpa	cpu: <unknown>/50%	1	5	1	# T+0

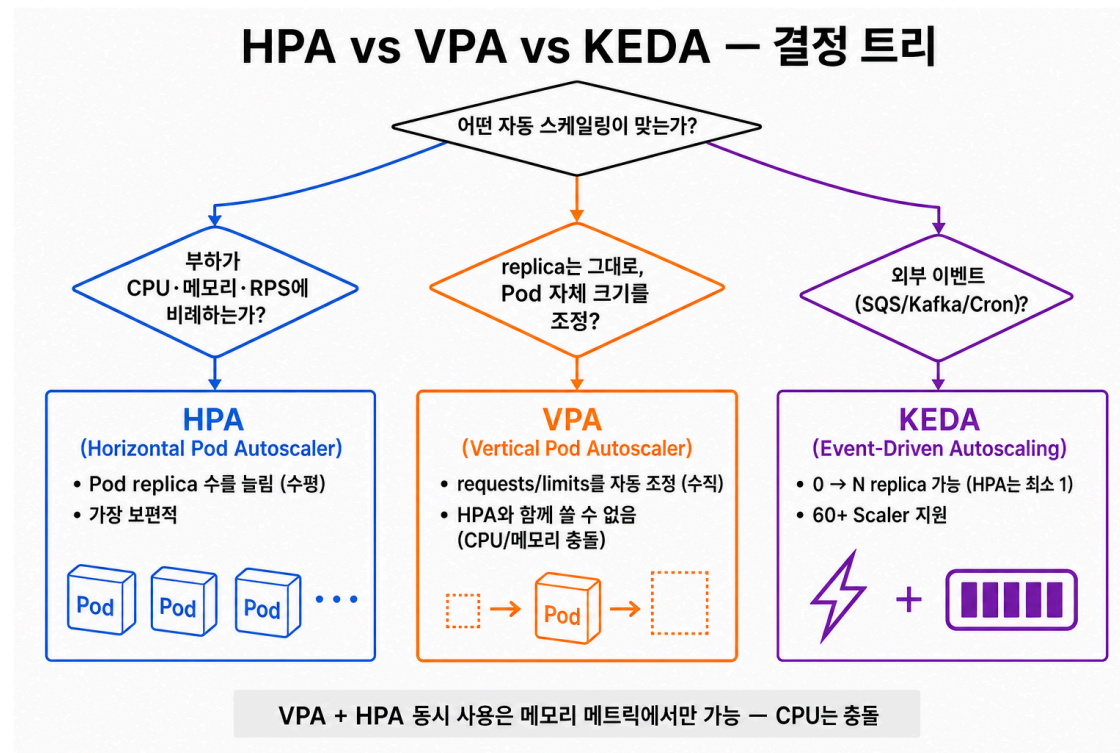
Demo 5 — HPA v2 스케일 결과 (실측 2/2)

```
# T+90s: metrics-server가 메트릭 도달 → behavior 정책에 따라 단계적 증가
$ kubectl get hpa cpu-stress-hpa
NAME          TARGETS          MINPODS  MAXPODS  REPLICAS          # ← 1 → 2 → 4
cpu-stress-hpa  cpu: 200%/50%    1         5         4

$ kubectl describe hpa cpu-stress-hpa | tail -8
Events:
  Warning   FailedGetResourceMetric   (T+0~T+15s)   no metrics returned          # 메트릭 도달 전
  Normal    SuccessfulRescale         T+60s         New size: 2
  Normal    SuccessfulRescale         T+75s         New size: 4
```

- behavior `scaleUp Percent 100 / 15s` → 1→2→4 단계적 증가 (15초 간격)
- `cpu 200%/50%` = request 200m 대비 400m 사용 = 4배 → 4 replica 권장
- `scaleDown stabilizationWindowSeconds: 300` → 부하 떨어져도 5분 후 감소 (flapping 방지)

14.6 HPA vs VPA vs KEDA — 결정 트리



3개 도구는 서로 다른 문제를 푼다: HPA = 부하 비례 → 수평 확장, VPA = 단일 Pod의 적정 크기 자동 추천/조정, KEDA = 외부 이벤트로 0→N (HPA 한계 우회). VPA + HPA는 메모리 메트릭에서만 같이 사용 가능 (CPU 충돌).

Demo 6 — VPA recommendation manifest

```
# 06-vpa-recommendation.yaml — 추천만, 자동 적용 안 함
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
spec:
  targetRef: { kind: Deployment, name: cpu-stress }
  updatePolicy:
    updateMode: "Off"          # "Off"=추천만 / "Auto"=Pod 재생성하며 적용
  resourcePolicy:
    containerPolicies:
      - containerName: '*'
        minAllowed: { cpu: "50m", memory: "32Mi" }
        maxAllowed: { cpu: "1", memory: "1Gi" }
```

```
$ kubectl get crd verticalpodautoscalers.autoscaling.k8s.io
Error from server (NotFound): VPA CRD 미설치
# → minikube에 VPA admission webhook 인증서 발급 까다로워 본 데모는 manifest 패턴만 학습
# → 운영 환경: github.com/kubernetes/autoscaler/vertical-pod-autoscaler 설치
```

recommendation 모드는 prod에서 안전 — 권장값을 보고 사람이 PR로 반영하는 게 표준

14.7 In-place Pod Resize — 1.27 alpha → 1.32 beta

```
# resizePolicy로 변경 시 동작 정의
spec:
  containers:
  - resources: { requests: { cpu: "100m" }, limits: { cpu: "200m" } }
    resizePolicy:
      - resourceName: cpu
        restartPolicy: NotRequired # cgroup 동적 변경 — Pod 재시작 없음
      - resourceName: memory
        restartPolicy: RestartContainer # 컨테이너만 재시작 (Pod IP·볼륨 유지)
```

변경의 핵심: 책 1판 시점에는 Pod resources 변경 = Pod 재생성. 1.27부터는 `kubectl patch ... --subresource resize` 로 재시작 없이 변경 가능. VPA `Auto` 모드의 동작도 in-place로 진화 중.

Demo 7 — In-place Resize 실측

```
$ kubectl apply -f 07-inplace-resize.yaml
$ kubectl get pod inplace-resize-pod -o jsonpath='{.spec.containers[0].resources}'
{"limits":{"cpu":"200m","memory":"128Mi"},
 "requests":{"cpu":"100m","memory":"64Mi"}}

# resize subresource로 CPU 100m → 300m, limits 200m → 500m
$ PATCH='{"spec":{"containers":[{"name":"main","resources":{"
  "requests":{"cpu":"300m","memory":"64Mi"},
  "limits":{"cpu":"500m","memory":"128Mi"}}}]}}'
$ kubectl patch pod inplace-resize-pod --subresource resize -p "$PATCH"
pod/inplace-resize-pod patched

$ kubectl get pod inplace-resize-pod
NAME                READY   STATUS    RESTARTS   AGE
inplace-resize-pod  1/1     Running   0           4m18s   # ← RESTARTS=0 !
```

- requests cpu 100m → 300m, limits cpu 200m → 500m로 변경 성공
- RESTARTS = 0 = Pod·컨테이너 재시작 없이 cgroup 동적 변경
- VPA Auto 모드가 안전해지는 기반 — 기존엔 Pod 재생성으로 다운타임 발생

Demo 8 — KEDA ScaledObject Cron Trigger (실측)

```
# 08-keda-scaledobject-cron.yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
spec:
  scaleTargetRef: { name: cron-worker }
  minReplicaCount: 0          # ← HPA로는 불가능, KEDA만 가능 (0→N)
  maxReplicaCount: 5
  triggers:
  - type: cron
    metadata:
      timezone: Asia/Seoul
      start: "* / 5 * * * *"    # 0,5,10... 분에 시작
      end: "1-59 / 5 * * * *"  # 1,6,11... 분에 종료 (1분간 active)
      desiredReplicas: "5"
```

```
$ kubectl get scaledobject cron-scaledobject
NAME                MIN  MAX  READY  ACTIVE  TRIGGERS  AGE
cron-scaledobject   0    5    True   False   cron      8s

$ kubectl get hpa | grep keda
keda-hpa-cron-scaledobject  <unknown>/1 (avg)  1  5  0  8s
```

KEDA는 내부적으로 HPA를 자동 생성 — 외부 메트릭(SQS·Kafka·cron)을 HPA가 이해하는 형태로 변환

Demo 9 — Pod-level Resources (1.32 alpha)

```
# 09-pod-level-resources.yaml - Pod 단위로 리소스 표현
apiVersion: v1
kind: Pod
spec:
  resources:                                # ← 1.32 alpha 신규 필드
    requests: { cpu: "300m", memory: "256Mi" }
    limits:   { cpu: "600m", memory: "512Mi" }
  containers:
  - { name: app,          image: busybox:stable, command: ["sleep","3600"] }
  - { name: sidecar-proxy, image: busybox:stable, command: ["sleep","3600"] }
# 컨테이너 단위 resources 생략 가능 - Pod이 자식들에게 동적 분배
```

```
$ kubectl apply -f 09-pod-level-resources.yaml
pod/pod-level-resources created
$ kubectl get pod pod-level-resources -o jsonpath='{.spec.resources}'
{"limits":{"cpu":"600m","memory":"512Mi"},"requests":{"cpu":"300m","memory":"256Mi"}}
$ kubectl get pod pod-level-resources
NAME                READY   STATUS    RESTARTS   AGE
pod-level-resources  2/2     Running   0           3s
```

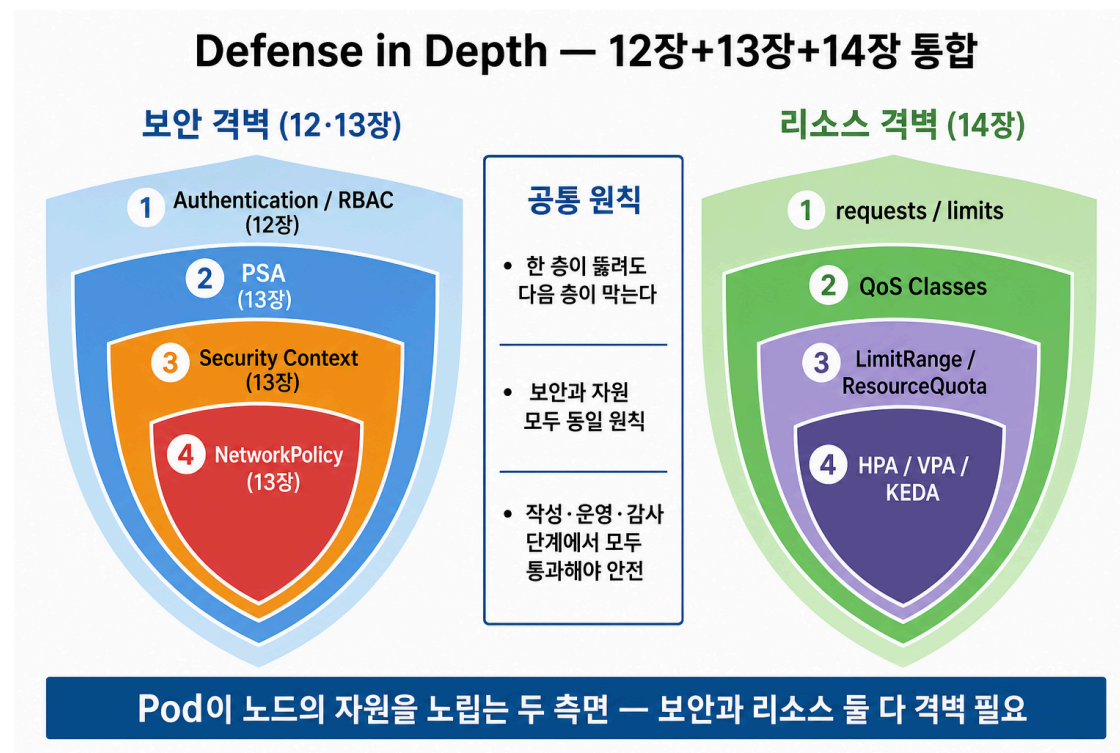
사이드카 시대의 답 — istio·linkerd 사이드카가 추가되어도 Pod 합계는 그대로

1판 책(2018) vs 현행(2026) — 14장 변경점

영역	책 1판	현행 (K8s 1.32+)
메트릭 수집	Heapster	metrics-server (1.13에서 Heapster 제거)
자동 스케일링 API	autoscaling/v1	autoscaling/v2 (1.23 GA)
HPA 메트릭	CPU 평균만	Resource + Pods + Object + External + ContainerResource
스케일 정책	즉시 반영	behavior 정책 (stabilizationWindow, Percent/Pods 정책)
수직 스케일러	없음	VPA (별도 설치) — recommendation/Auto
이벤트 기반	없음	KEDA — 0→N replica, 60+ Scaler
Pod resize	Pod 재생성 필요	--subresource resize (1.27 alpha → 1.32 beta)
자원 표현	컨테이너 단위만	Pod-level resources (1.32 alpha)
메모리 QoS	best-effort 강제	cgroup v2 MemoryQoS (1.27+) — memory.high 활용

책 14장의 YAML은 v1이라 동작하지만 단일 메트릭. 새로 쓰는 모든 HPA는 v2로 작성.

Defense in Depth — 12장+13장+14장 통합



두 격벽 동시 적용: 보안(12·13장)은 침해 차단, 자원(14장)은 노이즈 격리 + 비용 통제. 한 축만 켜면 한쪽으로 무너진다.

운영 체크리스트 — 새 namespace 만들 때 14장 + 13장

```
NS=production
kubectl create namespace ${NS}
kubectl label namespace ${NS} pod-security.kubernetes.io/enforce=restricted

# LimitRange + ResourceQuota를 GitOps Helm chart 한 묶음으로 적용
kubectl apply -n ${NS} -f - <<'EOF'
apiVersion: v1
kind: LimitRange
metadata: { name: container-limits }
spec:
  limits:
  - type: Container
    default: { cpu: "200m", memory: "256Mi" }
    defaultRequest: { cpu: "100m", memory: "128Mi" }
    max: { cpu: "2", memory: "2Gi" }
---
apiVersion: v1
kind: ResourceQuota
metadata: { name: ns-quota }
spec: { hard: { requests.cpu: "10", limits.cpu: "20", pods: "50" } }
EOF

# NetworkPolicy default-deny (13장)도 같은 chart에 포함
```

새 namespace 생성 직후 무조건 실행 — GitOps로 자동화

Summary — 14장 한 화면 정리

4축으로 Pod 자원을 다스린다

1. requests / limits — 컨테이너 단위, 스케줄링 기준 + cgroup 강제
2. QoS Classes — Guaranteed > Burstable > BestEffort 종료 우선순위
3. LimitRange / ResourceQuota — namespace 단위 default + 합계 한도
4. HPA / VPA / KEDA — 자동 스케일링 (수평 / 수직 / 이벤트)

오늘 실측한 것 (minikube + metrics-server + KEDA)

- Demo 1~4: requests/limits, QoS 3종 자동 결정, LimitRange 자동 부여, ResourceQuota admission denied
- Demo 5: HPA v2 부하 → cpu 200%/50% → 1→2→4 단계적 스케일 (behavior 정책)
- Demo 6: VPA manifest (CRD 미설치, 패턴 학습)
- Demo 7: In-place resize — RESTARTS=0으로 cpu 100m→300m 변경
- Demo 8: KEDA ScaledObject — 자동 HPA 생성 (keda-hpa-cron-scaledobject)
- Demo 9: Pod-level resources — `.spec.resources` 인식, 2/2 컨테이너 Running

기억할 단 한 줄

requests/limits는 보안의 자원 측면. 13장 NetworkPolicy처럼 default + 명시 모델. 명시 안 하면 BestEffort로 가장 먼저 죽는다.

질문 & 토론

자주 나오는 질문 3가지

- Q. requests를 크게 잡으면 더 빠르나?

A. 아니다. requests는 스케줄러 입력일 뿐, 실제 사용은 limits까지. 크게 잡으면 노드 빈 자리만 잡고 다른 Pod이 못 뜸 — 클러스터 자원 낭비.

- Q. limits 없이 가도 되나?

A. 안 된다. limits 없으면 BestEffort QoS가 되어 노드 자원 부족 시 가장 먼저 종료. 또한 노이즈 이웃이 노드 메모리 독점 가능 → 옆 Pod OOM.

- Q. HPA + VPA 같이 쓸 수 있나?

A. CPU 메트릭은 충돌 (HPA가 replica 늘리는데 VPA가 requests 키우면 양쪽 증폭). 메모리 메트릭에서만 같이 사용 가능. 일반 권장: HPA 단독 또는 VPA recommendation 모드 + 수동 PR.